MBARARA UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF COMPUTING AND INFORMATICS

BARCHELORS OF SCIENCE COMPUTER SCIENCE

CSC1204 DATABASE SYSTEMS

LECTURER	**MR. KENETH BAGUMA**
STUDENT	**WASENDA BLASIO MARVIN**
REG NUMBER	**2025/BCS/185/PS**
PROJECT DOMAIN	**OIL & GAS**
PROJECT CHOICE	**PETRO STATION DBMS**
GROUP NUMBER	**5**

1. Executive Summary

PetroStation DBMS is a database backed management system designed for a petrol station operating in the oil and gas retail domain. The project addresses operational problems such as manual records, revenue leakage, weak customer management, poor fuel inventory visibility, and delayed business reporting. The system provides an integrated database and web-based user interface for recording fuel sales, monitoring tanks, managing suppliers, handling fleet customer accounts, administering employees, tracking equipment maintenance, and producing management reports.

The implemented system uses a relational MySQL database and a PHP frontend deployed through XAMPP. The database design is based on an ER model that includes fuel types, tanks, pumps, sales transactions, customers, fleet accounts, suppliers, purchase orders, deliveries, invoices, employees, shifts, payroll, leave, assets, and maintenance logs. The frontend provides role-based access and allows the station manager to access all modules through a professional manager console.

The prototype demonstrates how a petrol station can move from manual or fragmented records to a structured information system. It improves accountability because every sale is linked to a pump, product, shift, attendant, and receipt number. It also improves inventory control because fuel sales reduce tank levels and deliveries increase tank levels. Procurement, HR, customer management, and maintenance are also represented in the system so that the database supports the station as a complete business operation.

The report concludes that the PetroStation DBMS is feasible and beneficial for the business case. The current prototype is appropriate for academic demonstration and can be expanded into a production ready system by adding full edit/delete workflows, audit trails, report exports, backup scheduling, and multi branch support.

2. Business Case

The business case for PetroStation DBMS is based on the need to improve operational control in a petrol station. Petrol stations handle fast-moving sales, stock-sensitive products, supplier deliveries, fleet credit customers, shift-based employees, and equipment that requires maintenance. When these activities are recorded manually or in isolated files, managers struggle to get reliable information in time to prevent loss or make good decisions.

The current issues include manual records, delayed summaries, weak accountability for attendants, revenue leakage, fuel variance, poor credit control, and lack of timely reporting. Manual records also create knowledge loss when staff leave or when documents are misplaced. The proposed solution is an integrated DBMS that consolidates all station operations in one database and presents them through a role-based frontend.

2.1 Business Objectives

- Reduce manual recordkeeping by storing operational data in a central database.
- Improve revenue accountability by linking sales to products, pumps, shifts, attendants, and receipts.
- Improve fuel stock control through tank levels, reorder thresholds, dip readings, and delivery records.
- Support customer and fleet account management, including credit limits and outstanding balances.
- Improve procurement control through suppliers, purchase orders, deliveries, and invoices.
- Support HR administration through employee records, shifts, attendance, payroll, and leave.
- Provide timely management information through dashboards, registers, and reports.

2.2 Expected Benefits

Benefit type	Explanation
Tangible benefits	Reduced fuel variance and theft, lower bad debt, labor savings, improved supplier negotiations, and better customer retention.
Intangible benefits	Better decision-making, customer trust, compliance, accountability, knowledge retention, and competitive positioning.
Financial expectation	The business case estimates UGX 12.3M in annual benefits, UGX 6.8M in one-time costs, UGX 2.21M in annual recurring costs, 240 percent ROI over three years, and an 18-month payback period.

3. Design: Logical Design

The logical design describes the data entities, relationships, and business rules independent of the physical database platform. The ER diagram models the station as related operational domains. Fuel types are stored in tanks, dispensed by pumps, and ordered from suppliers. Pumps are used in sales and assigned to employees during shifts. Customers may hold fleet accounts and may also be linked to sale transactions.

Procurement is modeled using suppliers, purchase orders, deliveries, and invoices. Human resource activities are modeled using employees, shifts, shift assignments, attendance records, payroll records, and leave requests. Equipment is modeled using assets and maintenance logs. These relationships ensure that station operations are connected instead of being stored as unrelated records.

3.1 Main ER Relationships

Relationship	Business meaning
FUEL_TYPE to TANK	A fuel product can be stored in one or more tanks.
FUEL_TYPE to PUMP	A fuel product can be dispensed by one or more pumps.
PUMP to SALE_TRANSACTION	Every fuel sale is recorded through a pump.
SHIFT and EMPLOYEE to SALE_TRANSACTION	Every sale is linked to a shift and attendant for accountability.
CUSTOMER to FLEET_ACCOUNT	A customer may hold one fleet credit account.
SUPPLIER to PURCHASE_ORDER	A supplier can fulfil many purchase orders.
PURCHASE_ORDER to DELIVERY	A purchase order can result in delivery records.
DELIVERY to INVOICE	A delivery can generate an invoice.
ASSET to MAINTENANCE_LOG	An asset can have many maintenance records.

3.2 Normalization

The design separates repeated concepts into their own tables. For example, fuel product details are stored in fuel_type and referenced by pumps, tanks, and purchase orders. Customer details are stored once in customer and referenced by sale_transaction and fleet_account. This reduces duplication, improves consistency, and supports referential integrity.

4. Design: Physical Design

The physical design was implemented in MySQL/MariaDB using SQL DDL statements. Each table has a primary key, appropriate data types, and foreign keys where relationships are required. Enumerated fields are used for statuses and controlled values such as payment mode, employee status, supplier status, and maintenance type.

The physical database is named petrostation_dbms. The schema file is stored as petrostation_schema.sql in the project folder. The frontend connects to the database using PDO and a local configuration file. In the current XAMPP setup, the project uses host 127.0.0.1 and port 3307 because another MySQL service was using the default port 3306.

4.1 Physical Table Groups

Group	Tables
Security	user_role, user_account
Fuel and inventory	fuel_type, tank, pump
Sales and customer management	customer, fleet_account, sale_transaction
Procurement	supplier, purchase_order, delivery, invoice
Human resources	employee, shift, shift_assignment, attendance, payroll, leave_request
Equipment and maintenance	asset, maintenance_log

4.2 Important Integrity Rules

- A tank must reference a valid fuel type.
- A pump must reference the fuel type it dispenses.
- A sale transaction must reference a pump, shift, and employee.
- A fleet account must reference a customer.
- A purchase order must reference a supplier, employee, and fuel type.
- A delivery must reference a purchase order and receiving tank.
- An invoice must reference a delivery and supplier.
- A maintenance log must reference an asset and technician.

5. Implementation: System Screenshots

The implemented system is a PHP web application running through XAMPP at http://localhost/petrostation/. The manager account has access to all modules. The screenshots below show the implemented user interface and confirm that the frontend exists beyond the database design.

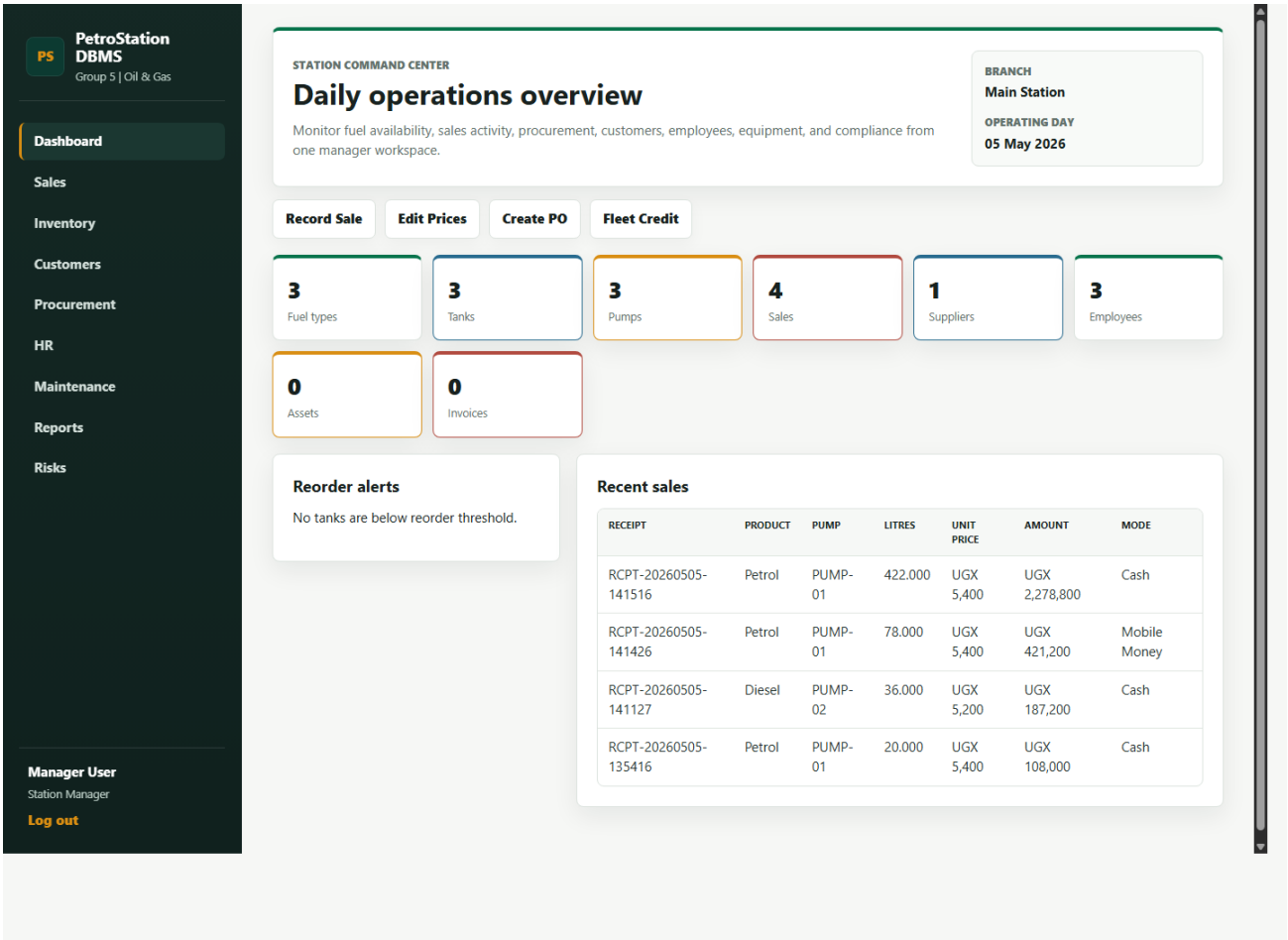

Figure 1: Manager dashboard showing KPI cards, quick actions, low-stock alerts, and recent sales.

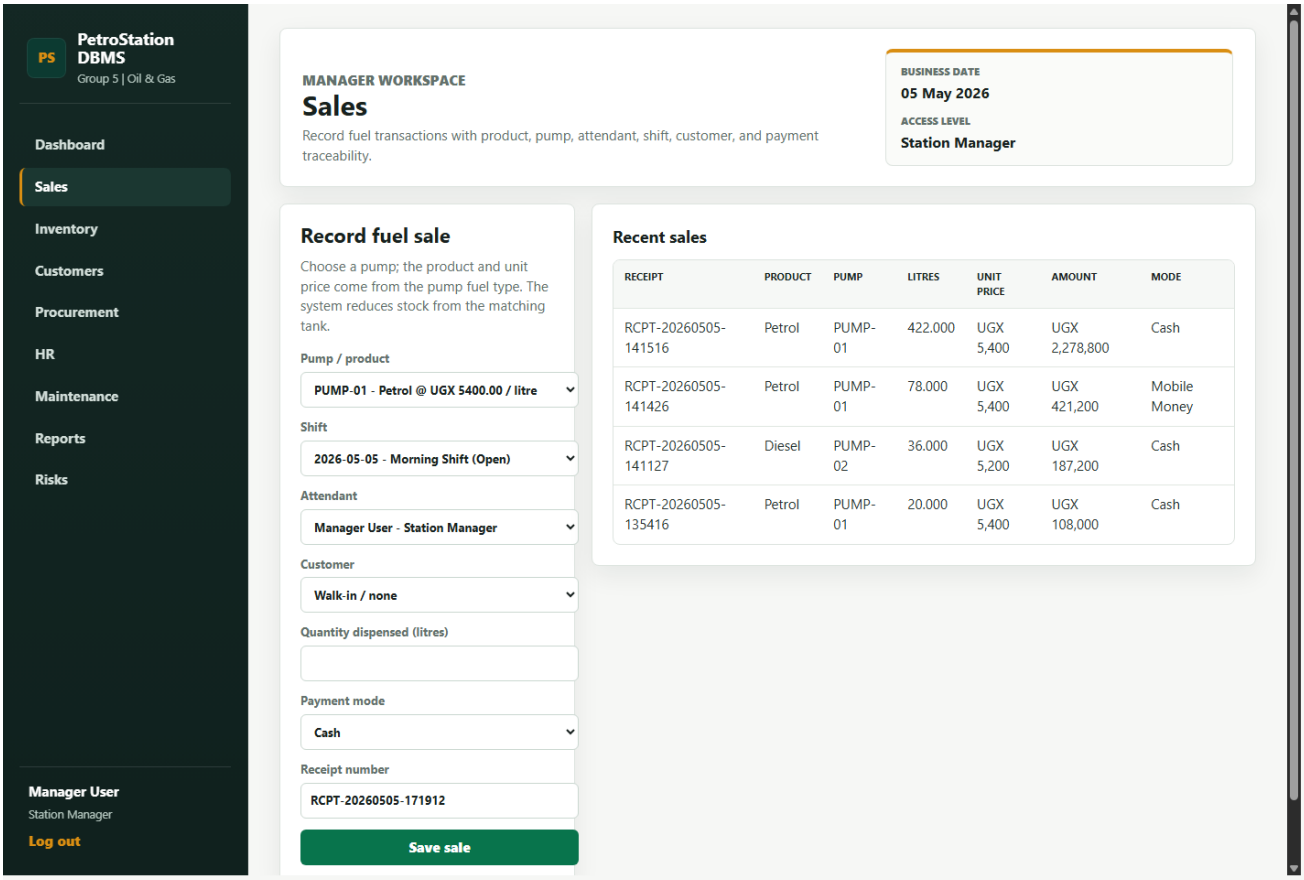

Figure 2: Sales module for recording a fuel transaction by pump/product, shift, attendant, customer, and payment mode.

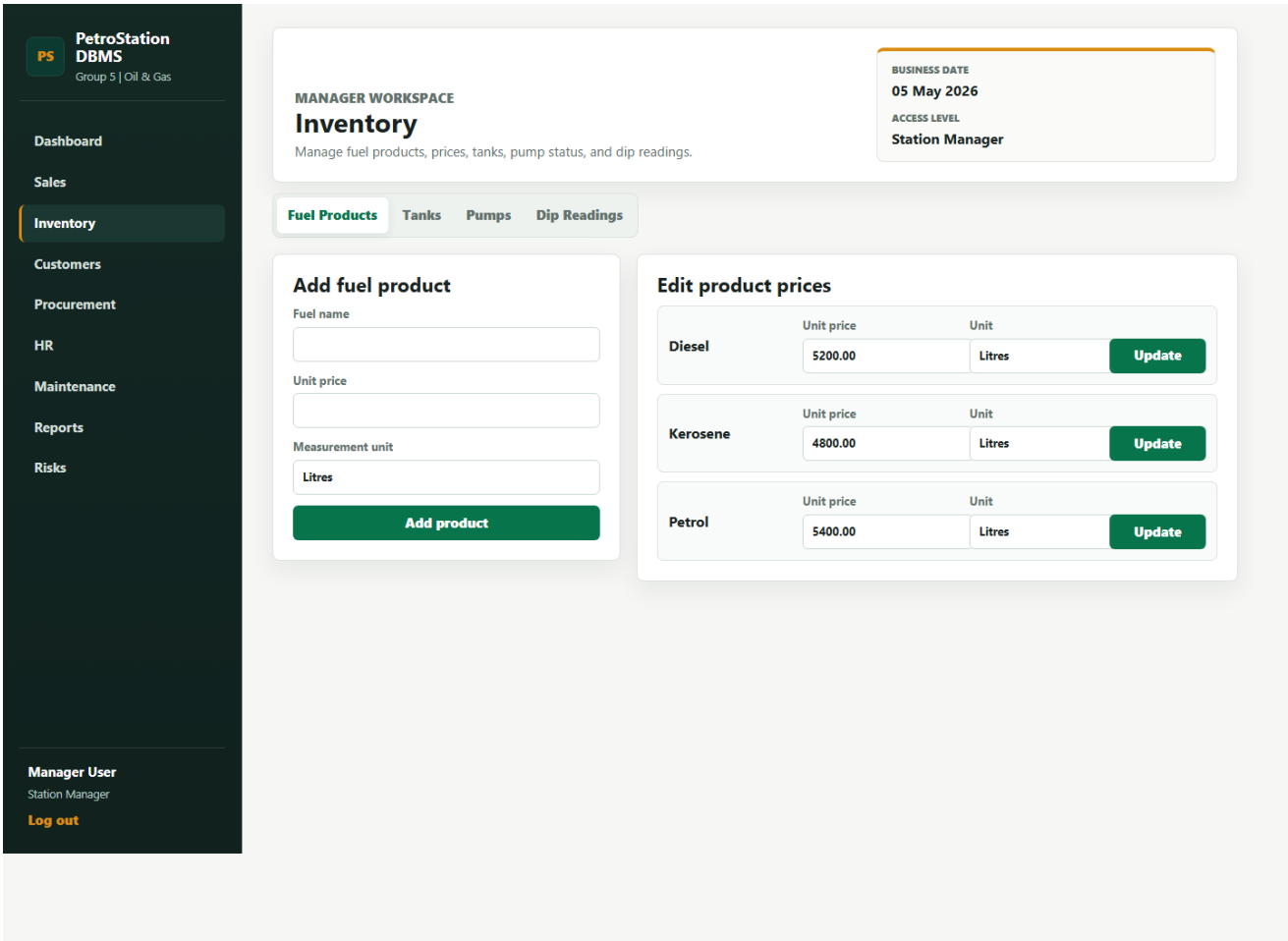

Figure 3: Inventory module showing fuel products and editable product prices.

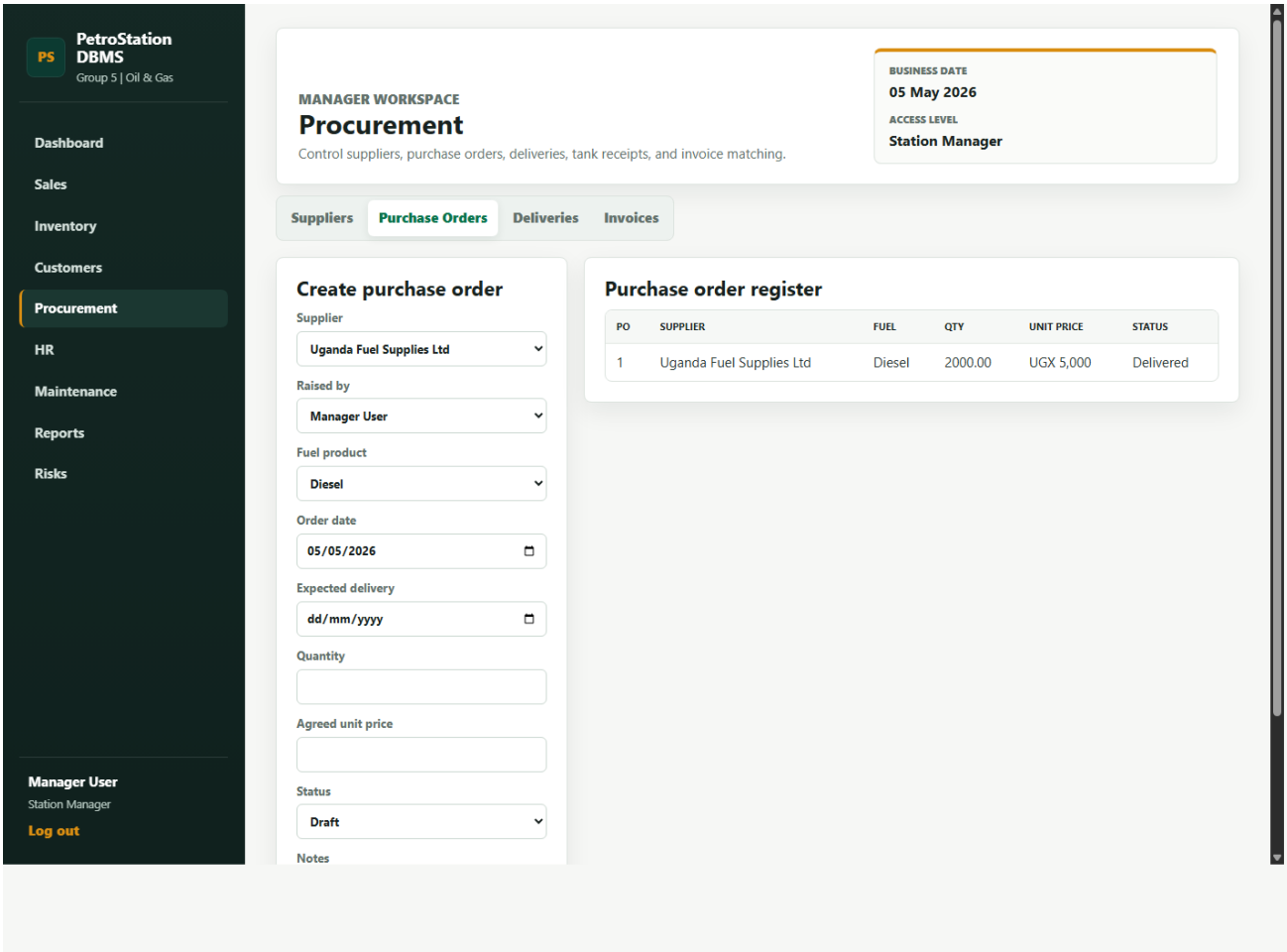

Figure 4: Procurement module showing purchase order entry and purchase order register.

6. Implementation: Table Descriptions

The following table describes the purpose of each table in the physical database. These tables correspond to the ER diagram and support the business functions in the frontend.

Table	Description
user_role	Stores the access roles used by the system, such as Station Manager, Pump Attendant, Cashier, Procurement Officer, and Owner.
user_account	Stores login credentials and connects each user to a role for role-based access control.
fuel_type	Defines fuel products sold by the station, including product name, selling price, and measurement unit.
tank	Stores fuel tank capacity, current level, reorder threshold, and the last dip reading date.
pump	Stores pump numbers, pump status, calibration dates, and the fuel product dispensed by each pump.
employee	Stores staff identity, role, phone number, national ID, salary, hire date, and employment status.
shift	Stores shift dates, start and end times, shift names, and operational status.
shift_assignment	Assigns employees to shifts and pumps and captures opening, closing, and variance readings.
attendance	Records employee attendance, check-in and check-out times, hours worked, and notes.
payroll	Stores monthly payroll records, gross salary, deductions, net salary, payment date, and payment status.
leave_request	Stores employee leave requests, leave type, date range, days count, approval status, and approver.
customer	Stores walk-in, individual, corporate, and fleet customer profiles with loyalty details.
fleet_account	Stores customer credit limits, outstanding balances, last statement date, and account status.
sale_transaction	Stores fuel sales by pump, shift, employee, customer, quantity, price, amount, payment mode, and receipt.
supplier	Stores supplier identity, contact details, performance rating, and status.
purchase_order	Stores fuel purchase orders raised to suppliers, including fuel product, quantity, price, expected delivery, and status.
delivery	Stores fuel deliveries received against purchase orders and tanks, including quantity, variance, driver, and delivery note.
invoice	Stores supplier invoices generated from deliveries, including amount, due date, payment status, and payment date.
asset	Stores station assets, linked pump where applicable, serial number, purchase value, and asset status.
maintenance_log	Stores servicing, repairs, calibration, inspection, cost, next service date, and compliance status for assets.

7. Implementation: Queries and Results

The queries below demonstrate that the database supports reporting and operational decision-making. They use joins, aggregation, conditional logic, and business fields from the implemented schema.

Query 1: Sales value by fuel product

SQL command:

```
SELECT ft.FuelName, COUNT(st.TransactionID) AS Transactions,
       COALESCE(SUM(st.QuantityDispensed),0) AS Litres,
       COALESCE(SUM(st.TotalAmount),0) AS SalesValue
FROM fuel_type ft
LEFT JOIN pump p ON p.FuelTypeID = ft.FuelTypeID
LEFT JOIN sale_transaction st ON st.PumpID = p.PumpID
GROUP BY ft.FuelTypeID, ft.FuelName
ORDER BY SalesValue DESC;
```

Query result:

FuelName	Transactions	Litres	SalesValue
Petrol	3	520.000	2808000.00
Diesel	1	36.000	187200.00
Kerosene	0	0.000	0.00

3 rows in set (0.003 sec)

Query 2: Tank stock and reorder status

SQL command:

```
SELECT t.TankLabel, ft.FuelName, t.CurrentLevel,
       t.ReorderThreshold,
       IF(t.CurrentLevel <= t.ReorderThreshold, 'Reorder', 'OK') AS Status
FROM tank t
JOIN fuel_type ft ON ft.FuelTypeID = t.FuelTypeID
ORDER BY t.TankLabel;
```

Query result:

TankLabel	FuelName	CurrentLevel	ReorderThreshold	Status
Tank A - Petrol	Petrol	21000.00	6000.00	OK
Tank B - Diesel	Diesel	14164.00	6000.00	OK
Tank C - Kerosene	Kerosene	4300.00	3500.00	OK

3 rows in set (0.002 sec)

Query 3: Fleet account credit exposure

SQL command:

```
SELECT c.CustomerName, fa.CreditLimit,
       fa.OutstandingBalance, fa.AccountStatus
FROM fleet_account fa
JOIN customer c ON c.CustomerID = fa.CustomerID;
```

Query result:

```
+--------------------+-------------+--------------------+---------------+
| CustomerName       | CreditLimit | OutstandingBalance | AccountStatus |
+--------------------+-------------+--------------------+---------------+
| Ruro Logistics Ltd |  5000000.00 |          850000.00 | Active        |
+--------------------+-------------+--------------------+---------------+
1 row in set (0.002 sec)
```

8. Implementation: Key Processes

8.1 Recording a Fuel Sale

1. The user selects an active pump from the sales form.
2. The system identifies the fuel product and unit price linked to that pump.
3. The user selects shift, attendant, customer, quantity, payment mode, and receipt number.
4. The system calculates total amount and inserts a sale_transaction record.
5. The system reduces the current level of the matching tank.
6. If the payment mode is Fleet Credit, the customer's fleet outstanding balance is increased.

8.2 Recording a Fuel Delivery

7. The user creates a purchase order for a supplier and fuel product.
8. The user records a delivery against the purchase order and receiving tank.
9. The system checks that the delivery will not exceed tank capacity.
10. The system inserts the delivery record and increases tank stock.
11. The purchase order status is updated to Delivered.
12. An invoice can then be recorded for the supplier and delivery.

8.3 Role-Based Access

Role-based access is implemented through user_role and user_account. The Station Manager can access all modules. Other users only access modules relevant to their responsibilities. This protects sensitive areas such as HR and procurement while still allowing operational users to perform their tasks.

9. Testing and Validation

Testing was performed by logging in as the manager user, opening the implemented modules, and verifying that pages loaded without PHP fatal errors. The sale workflow was tested to confirm that a sale records the selected pump/product and deducts stock from the related tank. Product price update was also tested to confirm that the frontend can update fuel prices.

Test area	Expected result	Observed result
Manager login	Manager can access all modules	Passed
Inventory price update	Fuel unit price can be edited	Passed
Sales entry	Sale records product, pump, shift, attendant, payment, and receipt	Passed
Tank stock update	Sale reduces stock and delivery increases stock	Passed
Procurement pages	Suppliers, purchase orders, deliveries, and invoices load	Passed
HR pages	Employees, shifts, attendance, payroll, and leave load	Passed
Maintenance pages	Asset and maintenance log pages load	Passed

9.1 Limitations Found During Testing

- The prototype supports adding and viewing most records, but full edit and delete controls are not yet available for every register.
- The system depends on manual entry of pump readings, tank dip readings, and accurate transaction data.
- Advanced reporting exports, predictive analytics, and automated bank reconciliation are not included.

10. Deployment and Access

The project was deployed locally using XAMPP. Apache serves the PHP frontend, and MySQL/MariaDB stores the database. The running copy is located in the XAMPP htdocs directory, while the editable source and schema are stored in the project folder.

Component	Location or setting
Project source folder	D:\MUST\YEAR_1-SEM_2\DATABASE SYSTEMS\Project
Frontend source	D:\MUST\YEAR_1-SEM_2\DATABASE SYSTEMS\Project\webapp
XAMPP running copy	C:\xampp\htdocs\petrostation
Schema file	D:\MUST\YEAR_1-SEM_2\DATABASE SYSTEMS\Project\petrostation_schema.sql
Database name	petrostation_dbms
Browser URL	http://localhost/petrostation/
Manager login	username: manager, password: admin123

10.1 Running the UI

13. Start Apache in the XAMPP Control Panel.
14. Start the XAMPP MySQL server. In this setup, MySQL uses port 3307.
15. Open a browser and go to http://localhost/petrostation/.
16. Log in using the manager demo account.
17. Use the sidebar to access sales, inventory, customers, procurement, HR, maintenance, reports, and risks.

11. Conclusion

The PetroStation DBMS demonstrates how a petrol station can use a relational database system to improve operational control and decision-making. The system connects sales, stock, procurement, customers, employees, and equipment maintenance in a single database-backed application. It satisfies the core business case by reducing manual records, improving accountability, supporting customer and supplier management, and providing timely reporting.

The logical and physical designs are aligned with the ER diagram, and the frontend provides implemented screens for the major business functions. The project is therefore a product of database design, SQL implementation, and frontend development. The recommended future work is to add complete edit/delete workflows, audit trails, report exports, backup scheduling, and multi-branch support.

12. Bibliography

Coronel, C., & Morris, S. (2019). Database Systems: Design, Implementation, & Management. Cengage Learning.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems. Pearson.

Oracle. (2026). MySQL 8.0 Reference Manual. Oracle Corporation.

Apache Friends. (2026). XAMPP Documentation. Apache Friends.

PHP Group. (2026). PHP Manual: PDO Database Connections. The PHP Documentation Group.

Project artifacts. (2026). PetroStation ER Diagram, Business Case, Schema SQL, and XAMPP frontend implementation.